Large Language Models

Yurii Paniv

24.10.2025

The Recap!

Word encoding

Vocabulary

index:	Word:
0	aardvark
1	able
...	...
2409	black
2410	bling
...	...
3202	candid
3203	cast
3204	cat
...	...
5281	is
5282	island
...	...
8676	the
8677	thing
...	...
9999	zombie

the cat is black

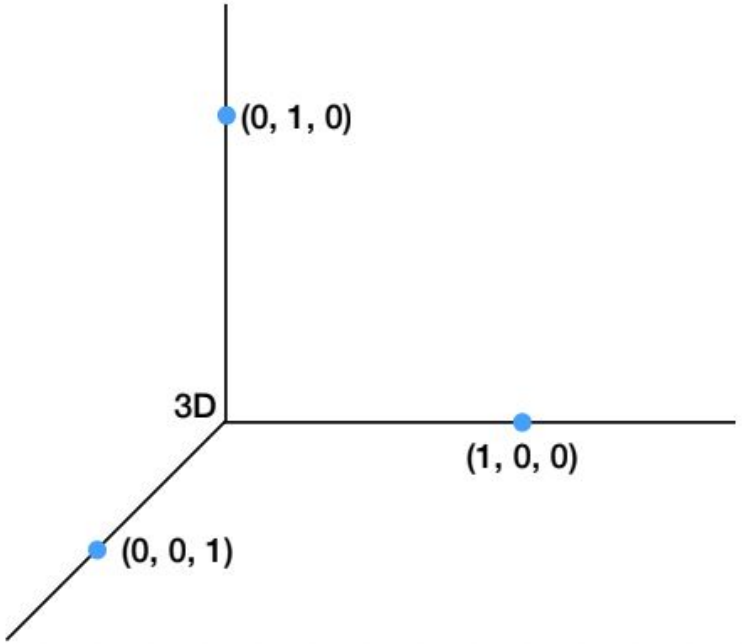
One-Hot Encoding

id	color
1	red
2	blue
3	green
4	blue

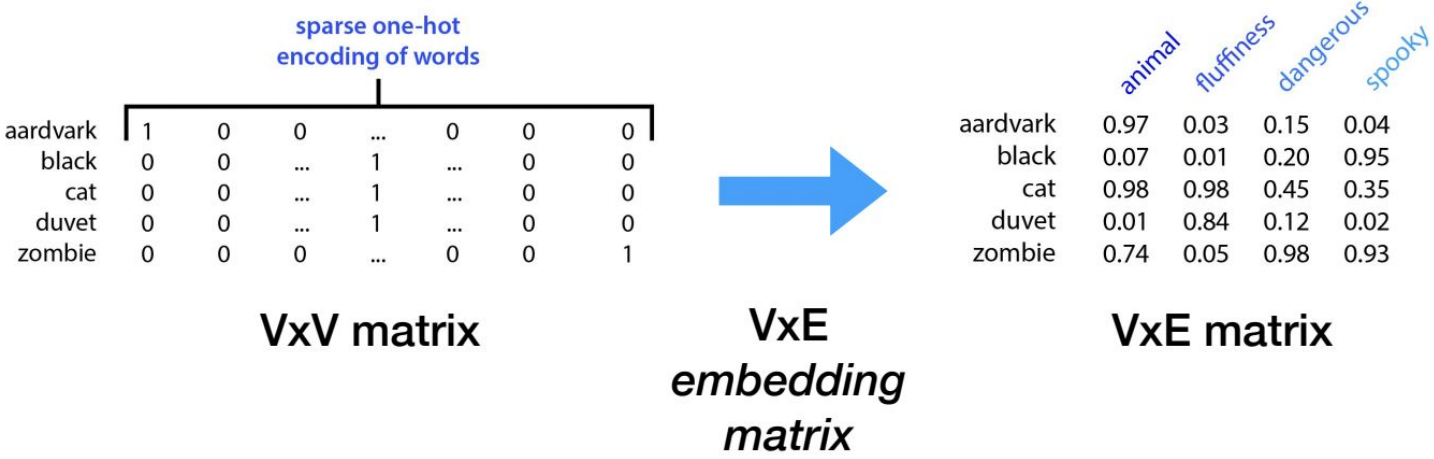
id	color_red	color_blue	color_green
1	1	0	0
2	0	1	0
3	0	0	1
4	0	1	0

What the problem with One-Hot Encoding

- Scales poorly with vocabulary size;
- Very high-dimensional sparse vectors;
- Violates what we know about word similarity .



Map one-hot to dense vector



But how to create this vector?

TF-IDF

$$w_{x,y} = tf_{x,y} \times \log\left(\frac{N}{df_x}\right)$$

TF-IDF

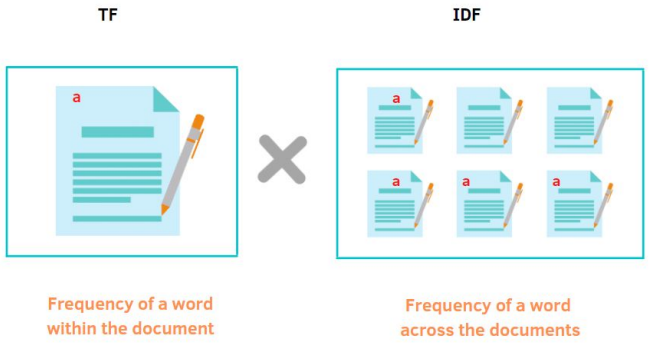
Term x within document y

$tf_{x,y}$ = frequency of x in y

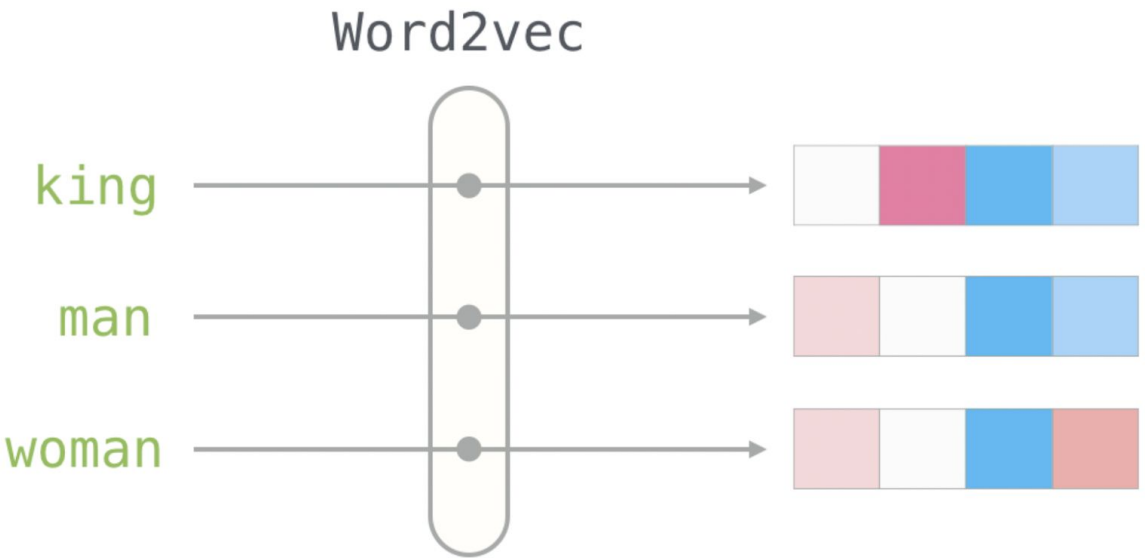
df_x = number of documents containing x

N = total number of documents

But this matrix is also sparse

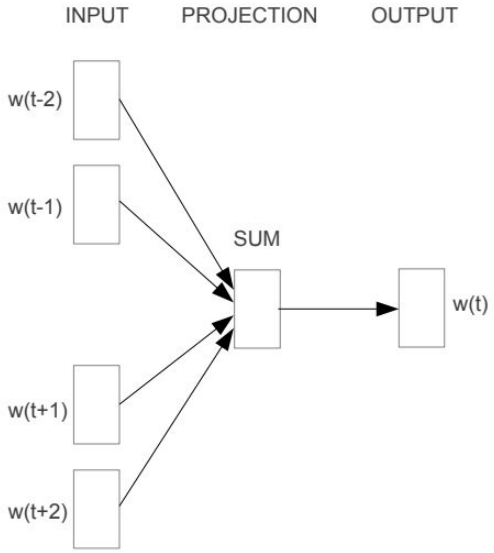


Word2Vec

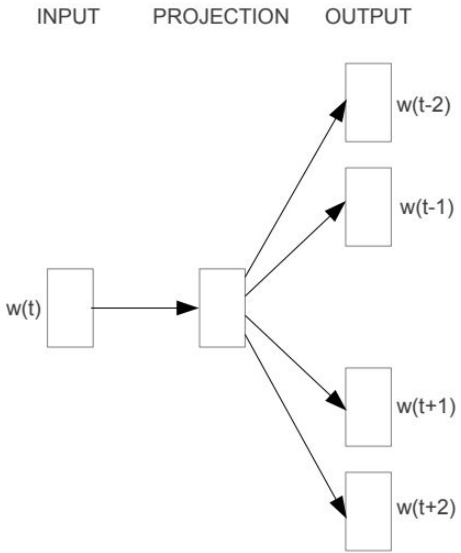


https://jalammar.github.io/illustrated-word2vec/

Word2Vec: Architectures



CBOW

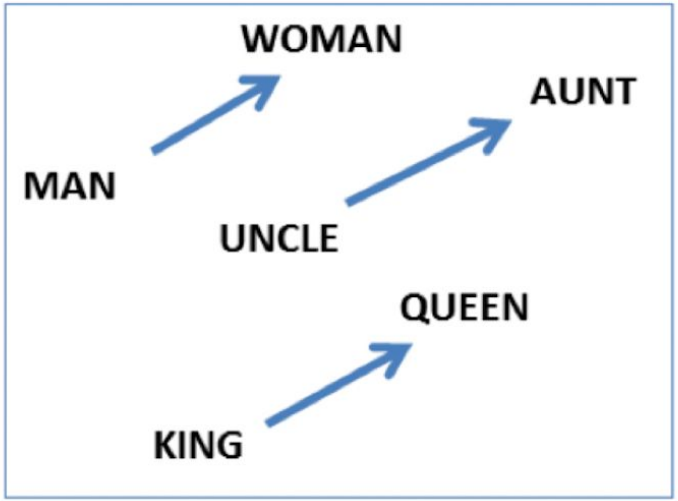


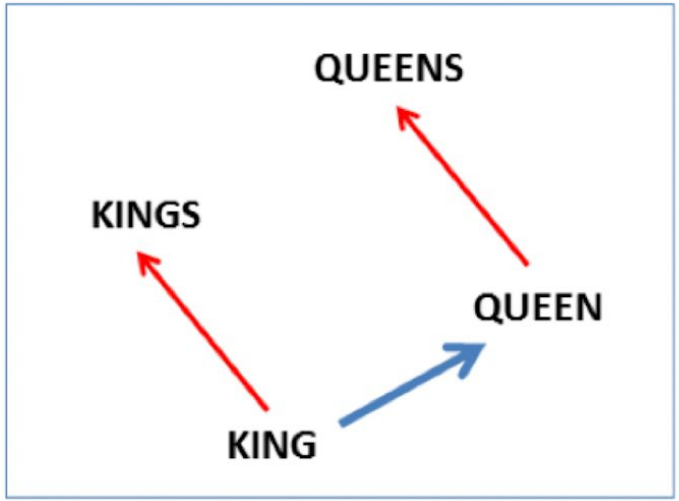
Skip-gram

Word2Vec

vector(*'king'*) - vector(*'man'*) + vector(*'woman'*) ≈ vector('queen')

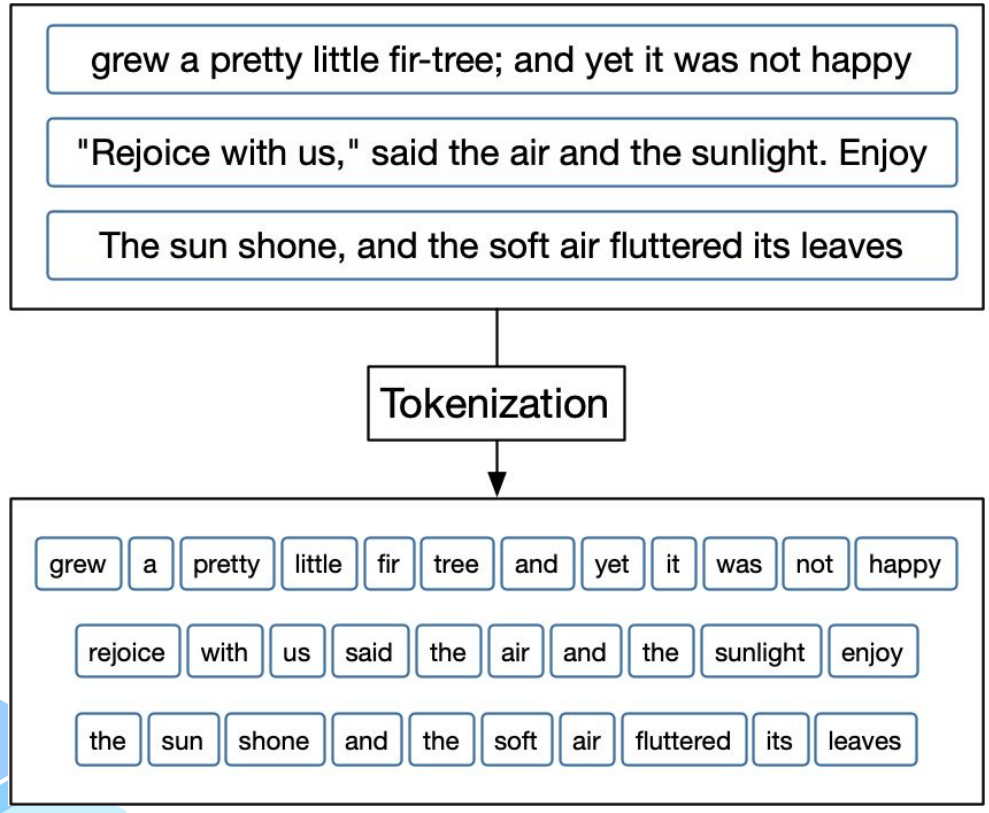
vector(*'Paris'*) - vector(*'France'*) + vector(*'Italy'*) ≈ vector('Rome')

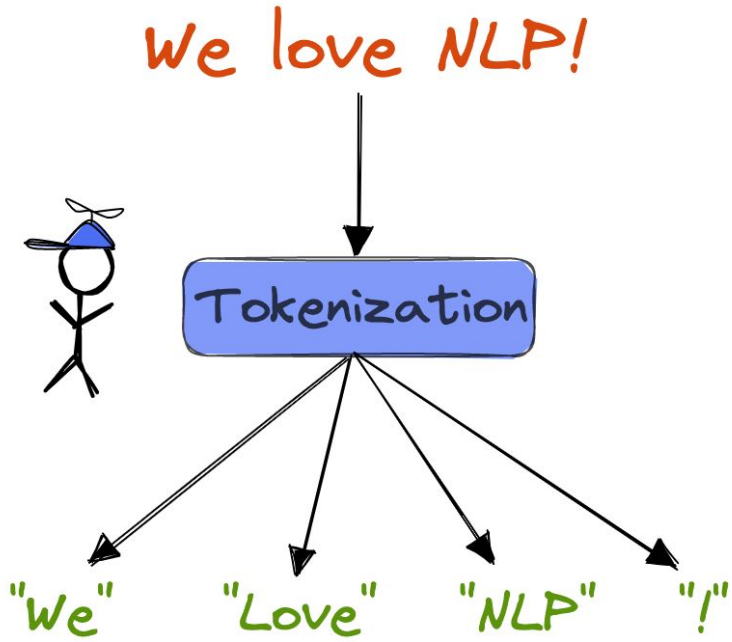




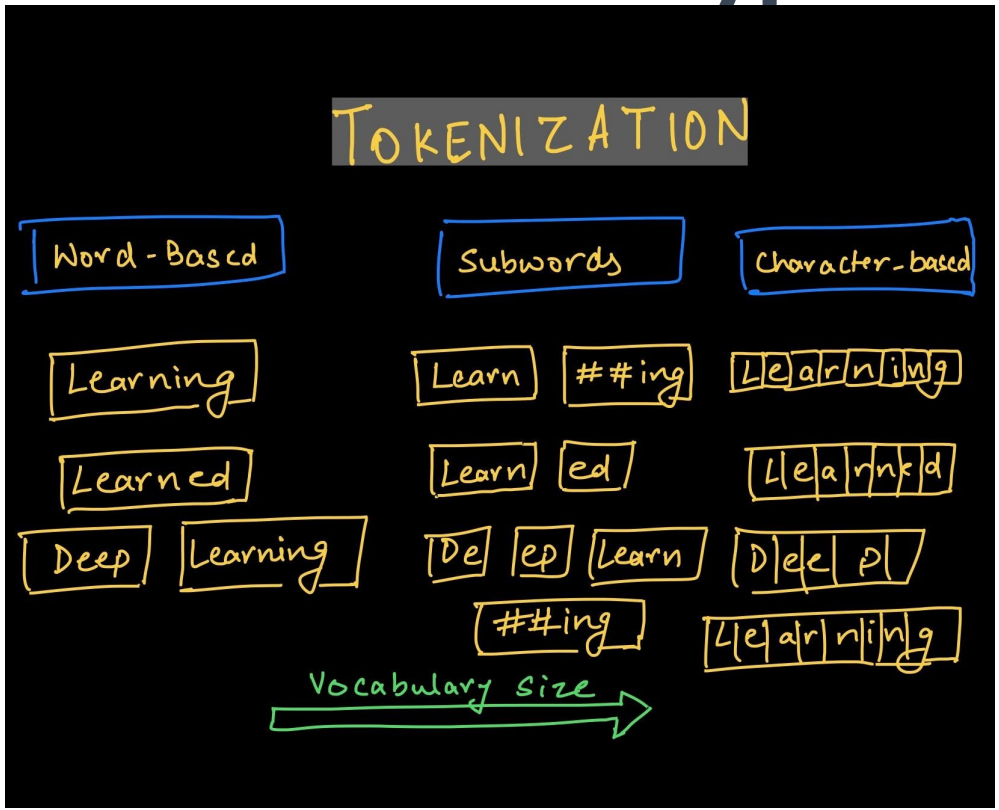
Step back: Tokenizer

Tokenization

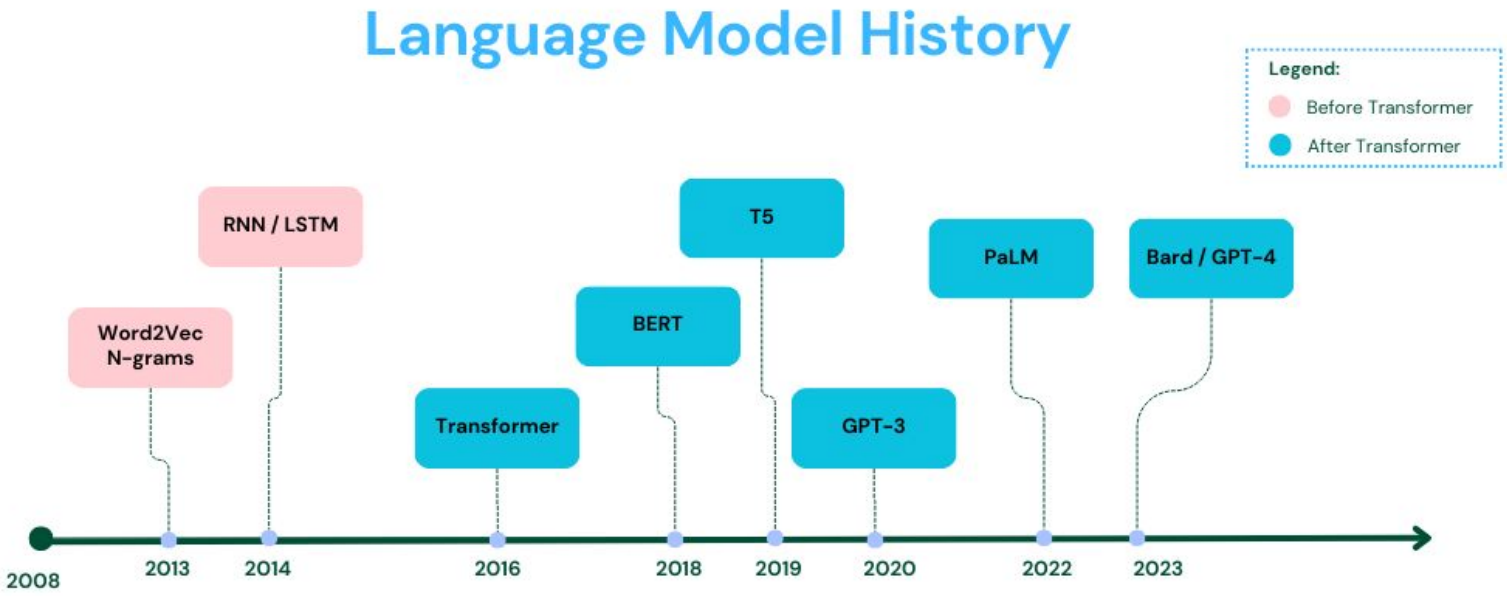




Tokenization Types



History of NLP models



LLMs? 😁

What is LLM?

What is LLM?

LLM stands for **Large Language Model**. It refers to a type of artificial intelligence (AI) model designed to understand, generate, and manipulate human language. These models are trained on vast amounts of text data and are capable of performing a wide range of language-related tasks, including answering questions, writing essays, summarizing text, translating languages, and even engaging in conversation.

How Large Language Models (LLMs)?

- Hardware improvements
- Transformer model architecture
- Data availability
- Self-supervised approach of pretraining

[Submitted on 18 Jan 2018 (v1), last revised 23 May 2018 (this version, v5)]

Universal Language Model Fine-tuning for Text Classification

Jeremy Howard, Sebastian Ruder

Inductive transfer learning has greatly impacted computer vision, but existing approaches in NLP still require task-specific modifications and training from scratch. We propose Universal Language Model Fine-tuning (ULMFiT), an effective transfer learning method that can be applied to any task in NLP, and introduce techniques that are key for fine-tuning a language model. Our method significantly outperforms the state-of-the-art on six text classification tasks, reducing the error by 18-24% on the majority of datasets. Furthermore, with only 100 labeled examples, it matches the performance of training from scratch on 100x more data. We open-source our pretrained models and code.

https://arxiv.org/abs/1801.06146

Language Models are Unsupervised Multitask Learners

Alec Radford [*1] **Jeffrey Wu** [*1] **Rewon Child** [1] **David Luan** [1] **Dario Amodei** [**1] **Ilya Sutskever** [**1]

Abstract

Natural language processing tasks, such as question answering, machine translation, reading comprehension, and summarization, are typically approached with supervised learning on task-specific datasets. We demonstrate that language models begin to learn these tasks without any explicit supervision when trained on a new dataset of millions of webpages called WebText. When

competent generalists. We would like to move towards more general systems which can perform many tasks – eventually without the need to manually create and label a training dataset for each one.

The dominant approach to creating ML systems is to collect a dataset of training examples demonstrating correct behavior for a desired task, train a system to imitate these behaviors, and then test its performance on independent and identically distributed (IID) held-out examples. This

https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

[Submitted on 28 May 2020 (v1), last revised 22 Jul 2020 (this version, v4)]

Language Models are Few-Shot Learners

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, Dario Amodei

Recent work has demonstrated substantial gains on many NLP tasks and benchmarks by pre-training on a large corpus of text followed by fine-tuning on a specific task. While typically task-agnostic in architecture, this method still requires task-specific fine-tuning datasets of thousands or tens of thousands of examples. By contrast, humans can generally perform a new language task from only a few examples or from simple instructions -

https://arxiv.org/abs/2005.14165

[Submitted on 27 Feb 2023]

LLaMA: Open and Efficient Foundation Language Models

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, Guillaume Lample

We introduce LLaMA, a collection of foundation language models ranging from 7B to 65B parameters. We train our models on trillions of tokens, and show that it is possible to train state-of-the-art models using publicly available datasets exclusively, without resorting to proprietary and inaccessible datasets. In particular, LLaMA-13B outperforms GPT-3 (175B) on most benchmarks, and LLaMA-65B is competitive with the best models, Chinchilla-70B and PaLM-540B. We release all our models to the research community.

https://arxiv.org/abs/2302.13971

[Submitted on 22 Jan 2025]

DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

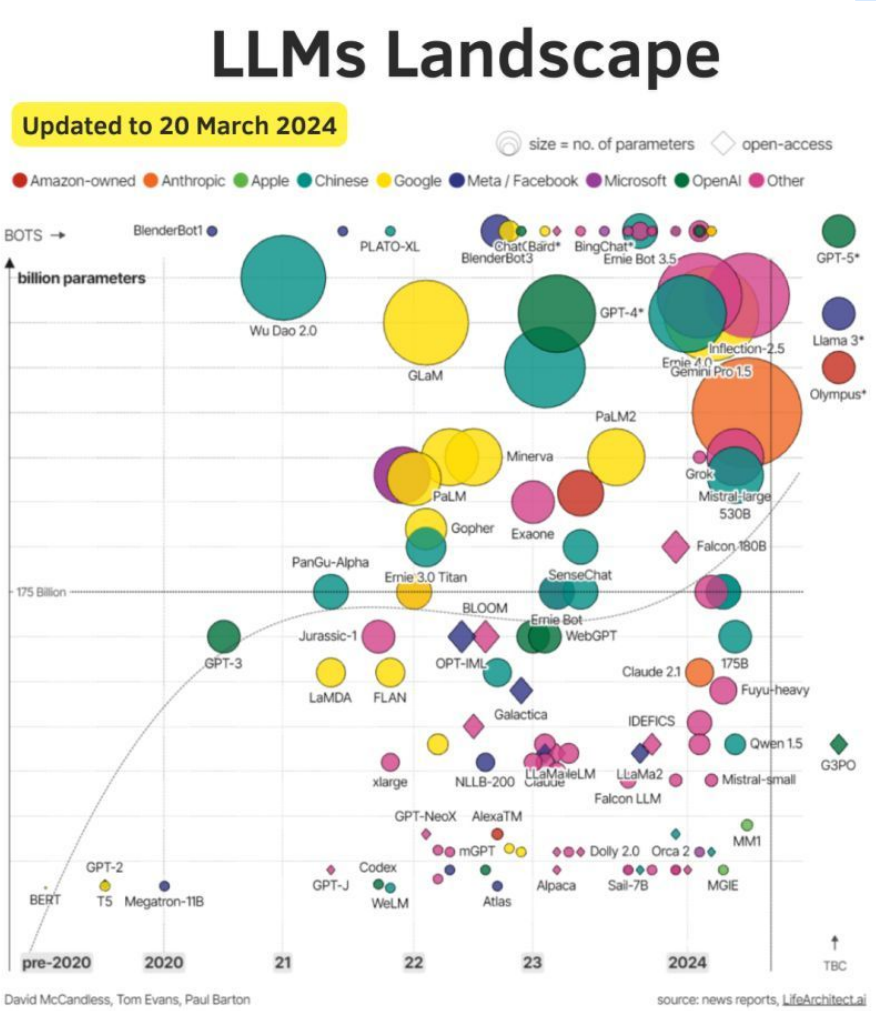
DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z.F. Wu et al. (100 additional authors not shown)

We introduce our first-generation reasoning models, DeepSeek-R1-Zero and DeepSeek-R1. DeepSeek-R1-Zero, a model trained via large-scale reinforcement learning (RL) without supervised fine-tuning (SFT) as a preliminary step, demonstrates remarkable reasoning capabilities. Through RL, DeepSeek-R1-Zero naturally emerges with numerous powerful and intriguing reasoning behaviors. However, it encounters challenges such as poor readability, and language mixing. To address these issues and further enhance reasoning performance, we introduce DeepSeek-R1, which incorporates multi-stage training and cold-start data before RL. DeepSeek-R1 achieves performance comparable to OpenAI-o1-1217 on reasoning tasks. To support the research community, we open-source DeepSeek-R1-Zero, DeepSeek-R1, and six dense models (1.5B, 7B, 8B, 14B, 32B, 70B) distilled from DeepSeek-R1 based on Qwen and Llama.

Subjects: **Computation and Language (cs.CL)**; Artificial Intelligence (cs.AI); Machine Learning (cs.LG)

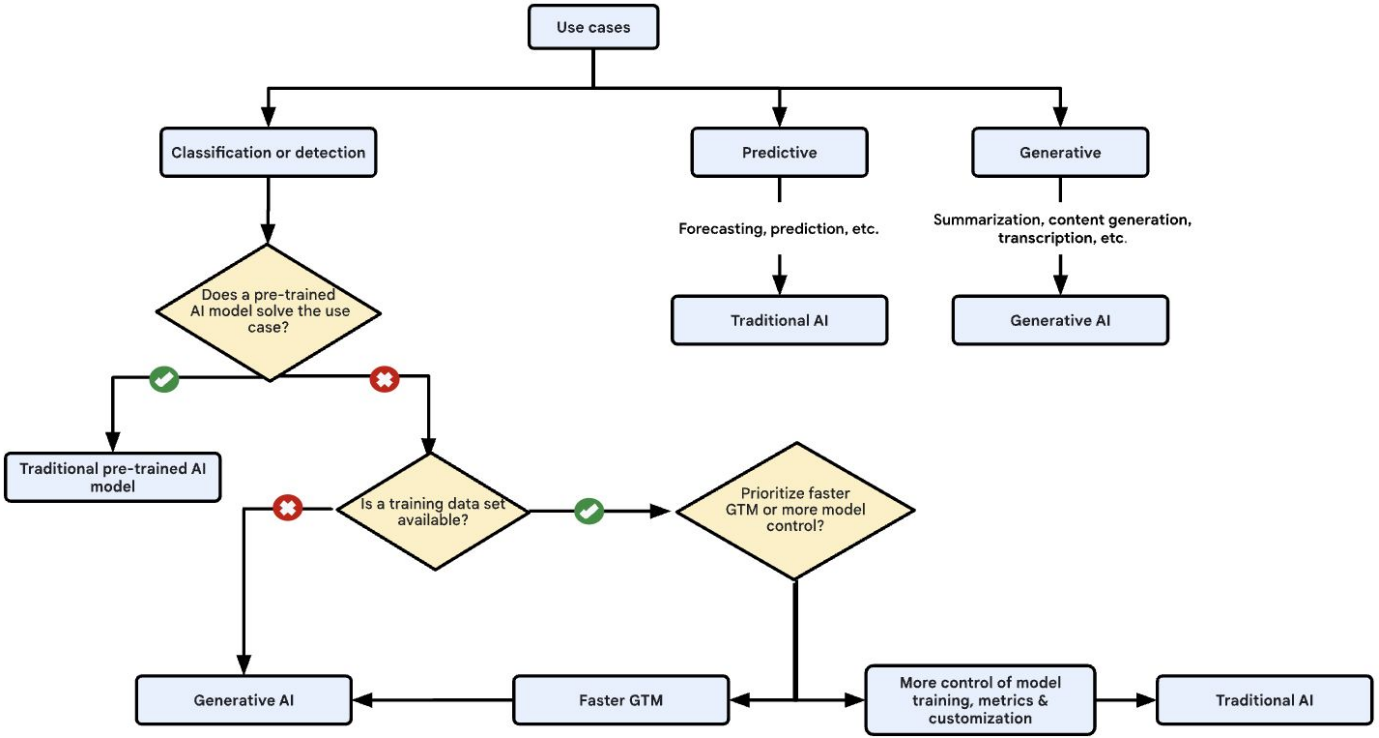
https://arxiv.org/abs/2501.12948

A lot of LLMs

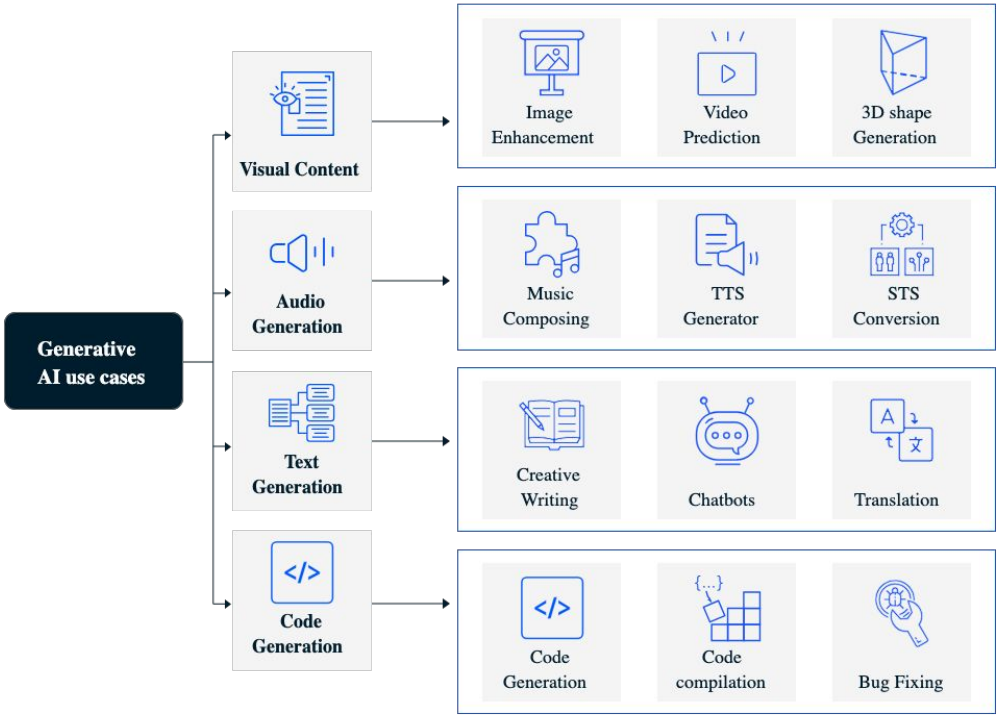

LLMs Landscape
Updated to 20 March 2024
size = no. of parameters
open-access
Amazon-owned
Anthropic
Apple
Chinese
Google
Meta / Facebook
Microsoft
OpenAI
Other
BOTS
billion parameters
175 Billion
BlenderBot1
PLATO-XL
ChatGPT
Bard*
BingChat*
BlenderBot3
Ernie Bot 3.5
GPT-5*
Wu Dao 2.0
GLaM
GPT-4*
Llama 3*
Inflection-2.5
Ernie 4.0
Gemini Pro 1.5
Olympus*
PaLM2
Minerva
Grok
PaLM
Mistral-large
530B
Gopher
Exaone
Falcon 180B
PanGu-Alpha
Ernie 3.0 Titan
SenseChat
BLOOM
Ernie Bot
Jurassic-1
WebGPT
GPT-3
OPT-IML
Claude 2.1
175B
LaMDA
FLAN
Fuyu-heavy
Galactica
IDEFICS
Qwen 1.5
G3PO
xlarge
NLLB-200
LLaMa2
Mistral-small
Falcon LLM
GPT-NeoX
AlexaTM
MM1
mGPT
Dolly 2.0
Orca 2
GPT-2
Codex
GPT-J
BERT
T5
Megatron-11B
WeLM
Atlas
Alpaca
Sail-7B
MGIE
pre-2020
2020
21
22
23
2024
TBC
David McCandless, Tom Evans, Paul Barton
source: news reports, LifeArchitect.ai

Real-World Use-Cases: Downstream tasks

Decide between traditional AI and



Common Generative AI Use Cases



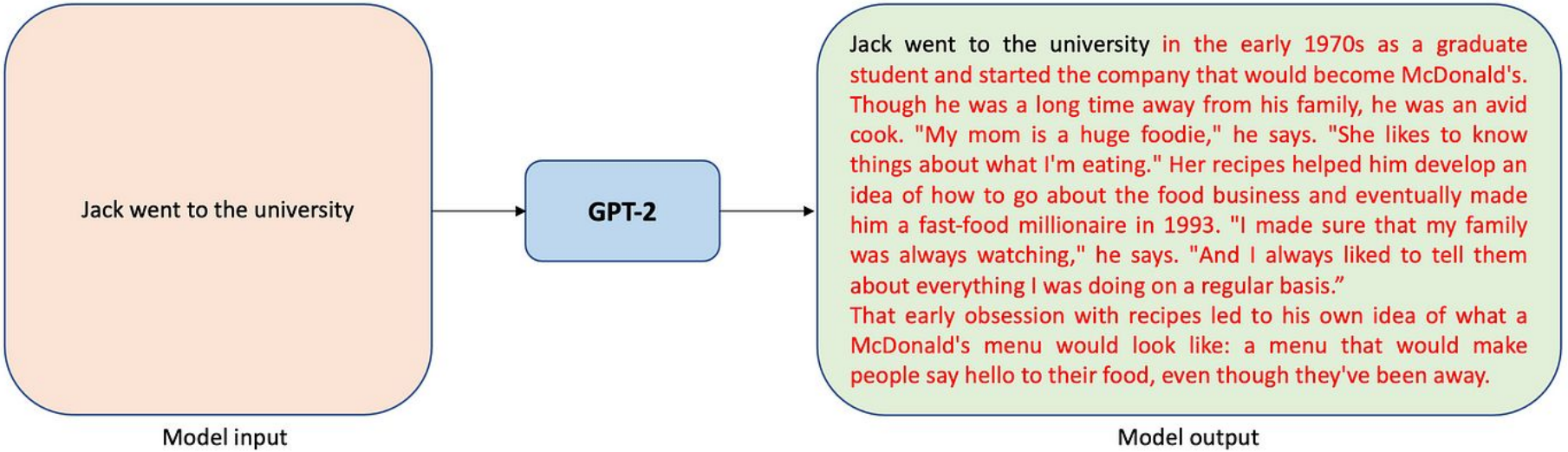
LeewayHertz

Common LLMs Use Cases

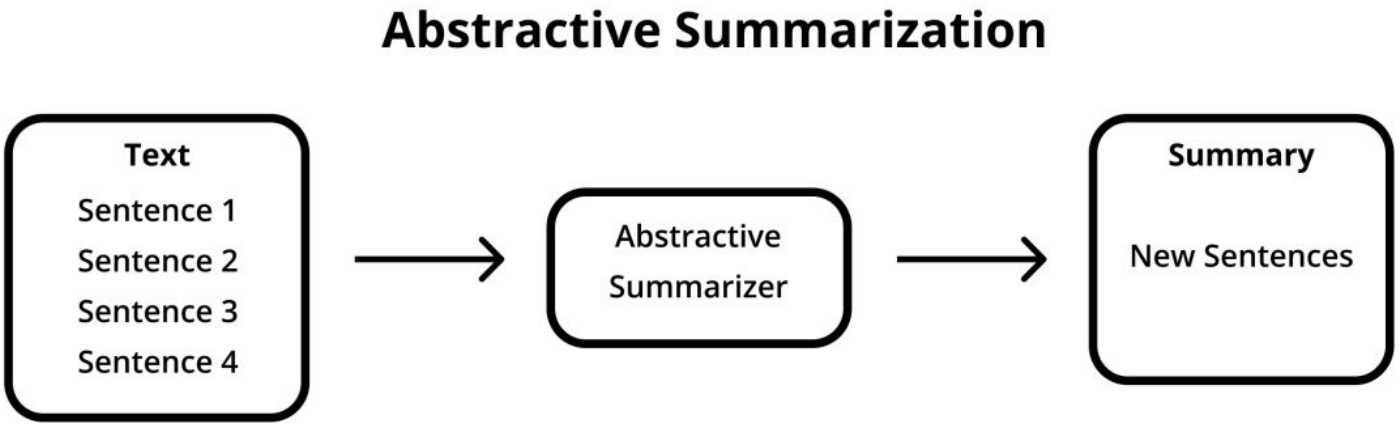
Common tasks LLMs are trained to solve

Clockwise Software

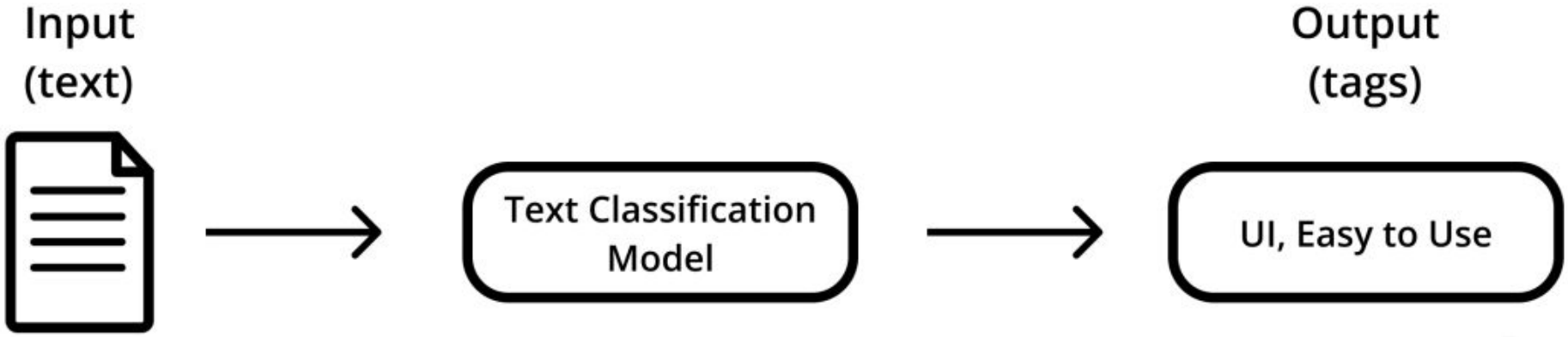
Text Generation



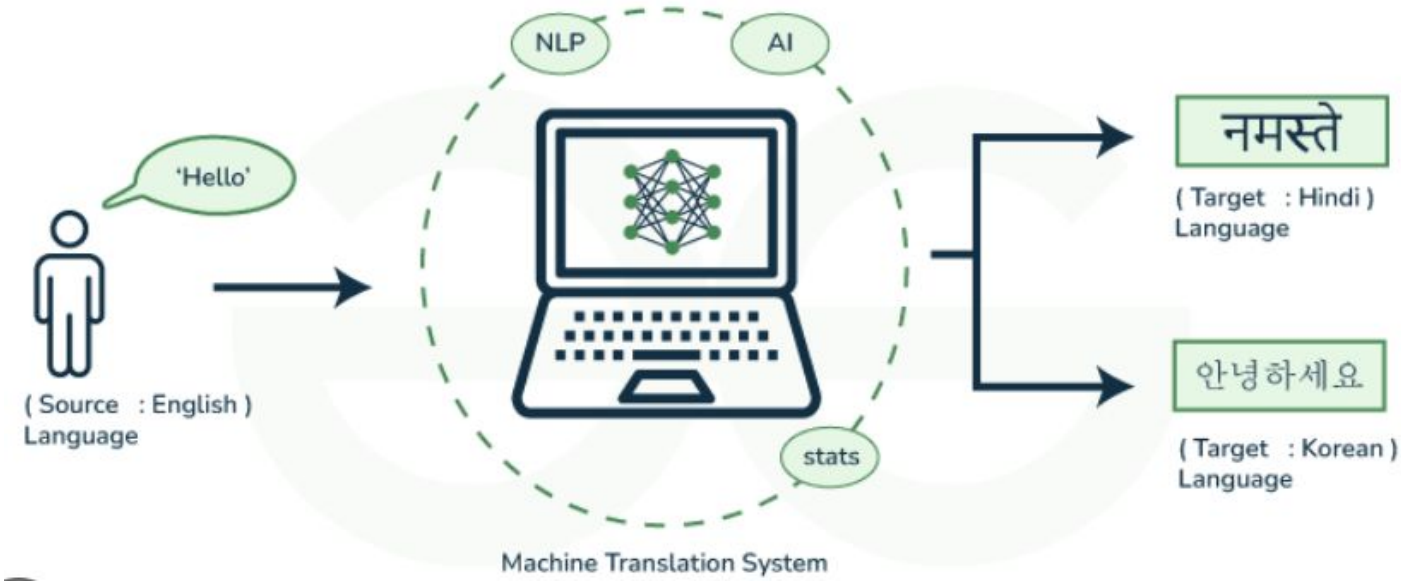
Summarization



Text Classification



Text Translation



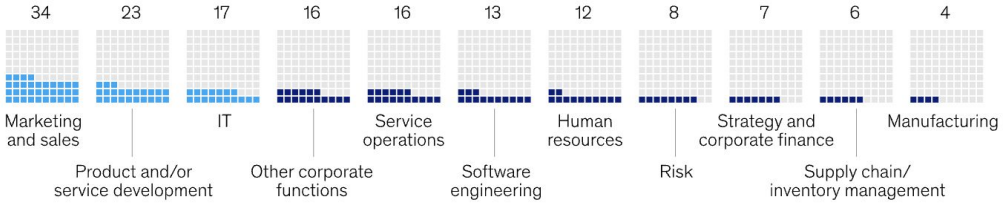
Grammatical Error Correction

Input (Erroneous)	Output (Corrected)
A important part of my life have been a people that stood by me.	An important part of my life has been the people who stood by me.

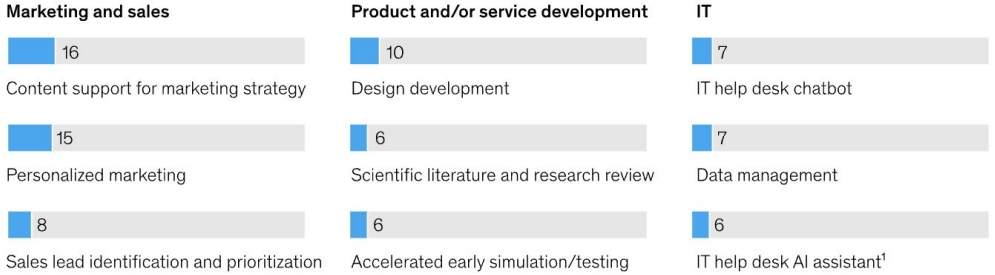
Companies popular Use

Respondents most often report generative AI adoption in their marketing-and-sales, product- and service-development, and IT functions.

Respondents' organizations regularly using generative AI (gen AI), by function, % of respondents



Most commonly reported gen AI use cases within function, % of respondents

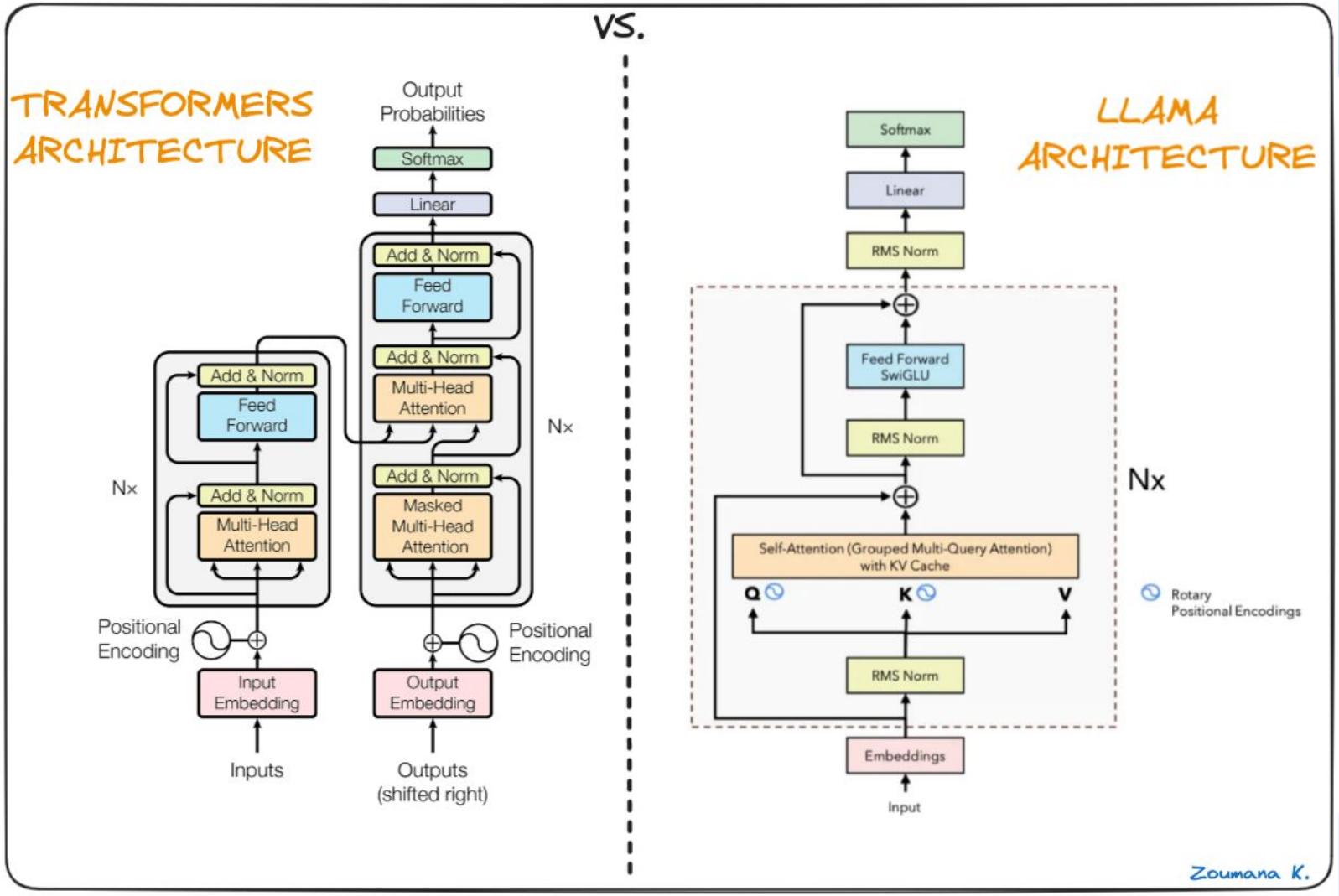


[1]Eg, providing real-time assistance and script suggestions to help desk employees during human-to-human conversations.
Source: McKinsey Global Survey on AI, 1,363 participants at all levels of the organization, Feb 22–Mar 5, 2024

LLM Do's and Don'ts

	Common AI Techniques					
Use Case Families	Generative Models	Non-Generative ML	Optimisation	Simulation	Rules	Graphs
Forecasting	Low	High	Low	High	Medium	Low
Planning	Low	Low	High	Medium	Medium	High
Decision Intelligence	Low	Medium	High	High	High	Medium
Autonomous System	Low	Medium	High	Medium	Medium	Low
Segmentation	Medium	High	Low	Low	High	High
Recommender	Medium	High	Medium	Low	Medium	High
Perception	Medium	High	Low	Low	Low	Low
Intelligent Automation	Medium	High	Low	Low	High	Medium
Anomaly Detection	Medium	High	Low	Medium	Medium	High
Content Generation	High	Low	Low	High	Low	Low
Chatbots	High	High	Low	Low	Medium	High
Knowledge Discovery	High	Medium	Low	Low	Medium	High

https://pub.towardsai.net/do-not-use-llm-or-generative-ai-for-these-use-cases-a819ae2d9779


VS.
TRANSFORMERS ARCHITECTURE
Output Probabilities
Softmax
Linear
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Add & Norm
Masked Multi-Head Attention
Nx
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Nx
Positional Encoding
Positional Encoding
Input Embedding
Output Embedding
Inputs
Outputs (shifted right)
LLAMA ARCHITECTURE
Softmax
Linear
RMS Norm
Feed Forward SwiGLU
RMS Norm
Self-Attention (Grouped Multi-Query Attention) with KV Cache
Q
K
V
RMS Norm
Embeddings
Input
Nx
Rotary Positional Encodings
Zoumana K.

Next token is always a probability

Enter text:

One, two,

3198 11 734 11

Prediction

#	probs	next token ID	predicted next token
0	39.71%	1115	three
1	16.97%	290	and
2	7.55%	734	two
3	3.76%	1440	four
4	2.76%	393	or
5	2.18%	1936	five
6	1.57%	530	one
7	1.43%	345	you
8	1.15%	257	a
9	0.84%	3598	seven

Which we can use!

Enter text:

One, two, three, four, mango

One , two , three , four , mango

#	probs	predicted next token
0	0.00%	mango
1	84.64%	five
2	1.63%	ten
3	1.29%	Five
4	1.07%	six
5	0.80%	four
6	0.73%	one
7	0.69%	and
8	0.50%	fifty
9	0.50%	maybe
10	0.49%	seven

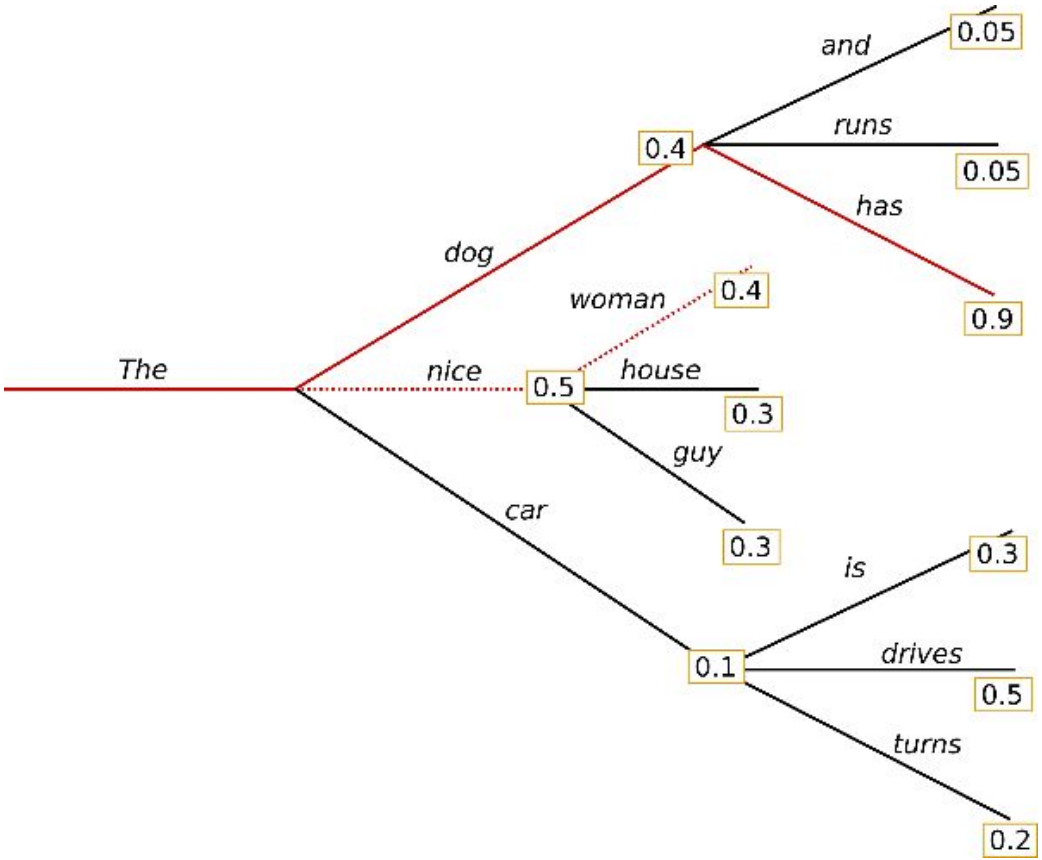
Generation Parameters

- beam-search
- sampling

Beam Search

- num_beams (int, optional, defaults to 1) — Number of beams for beam search. 1 means no beam search.
- temperature (float, optional, defaults to 1.0) — The value used to module the next token probabilities.

Beam Search


and
0.05
runs
0.4
0.05
has
0.9
dog
woman
0.4
The
nice
0.5
house
0.3
guy
0.3
car
is
0.3
0.1
drives
0.5
turns
0.2

Use cases

Use cases where accuracy is critical, like translation.

Downsides:

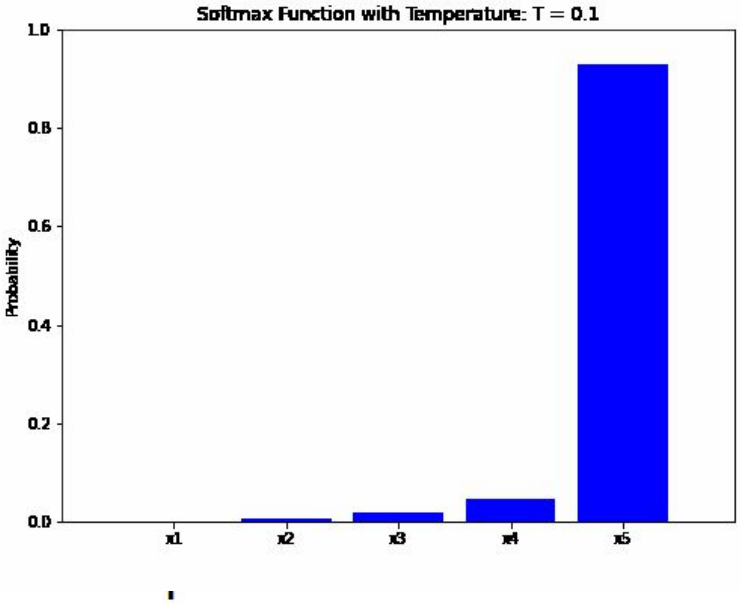
- computationally and memory expensive
- undertrained models can repeat tokens infinitely

Sampling - Probability top_k and top_p

- do_sample (bool, optional, defaults to False) — Whether or not to use sampling ; use greedy decoding otherwise.
- temperature (float, optional, defaults to 1.0) — The value used to module the next token probabilities.
- top_k (int, optional, defaults to 50) — The number of highest probability vocabulary tokens to keep for top-k-filtering.
- top_p (float, optional, defaults to 1.0) — If set to float < 1, only the most probable tokens with probabilities that add up to top_p or higher are kept for generation.

Temperature

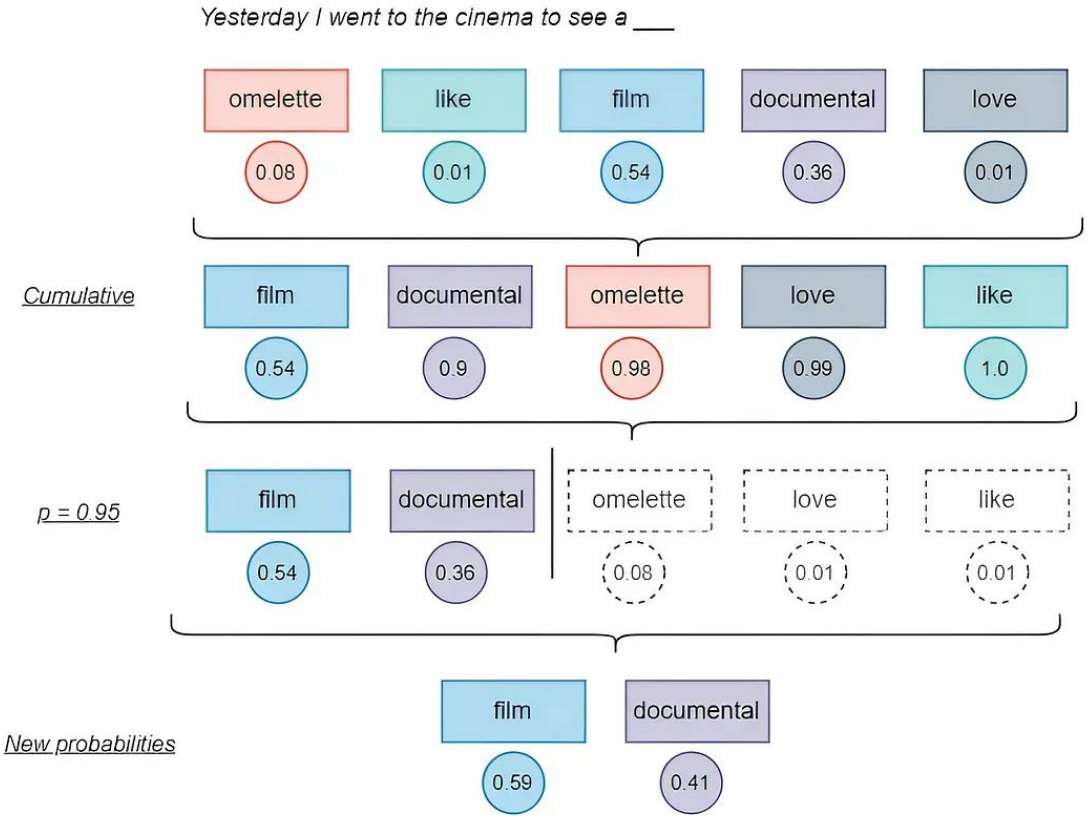
Softmax takes a vector of real numbers as input and normalizes it into a probability distribution.
The temperature parameter is introduced in a softmax function to control the “softness” or “peakiness” of the output probability distribution. The temperature is a parameter that we use to control the level of randomness in the function’s output.



$$softmax(y_i) = \frac{e^{\frac{y_i}{T}}}{\sum_{j=1}^{n} e^{\frac{y_j}{T}}}$$

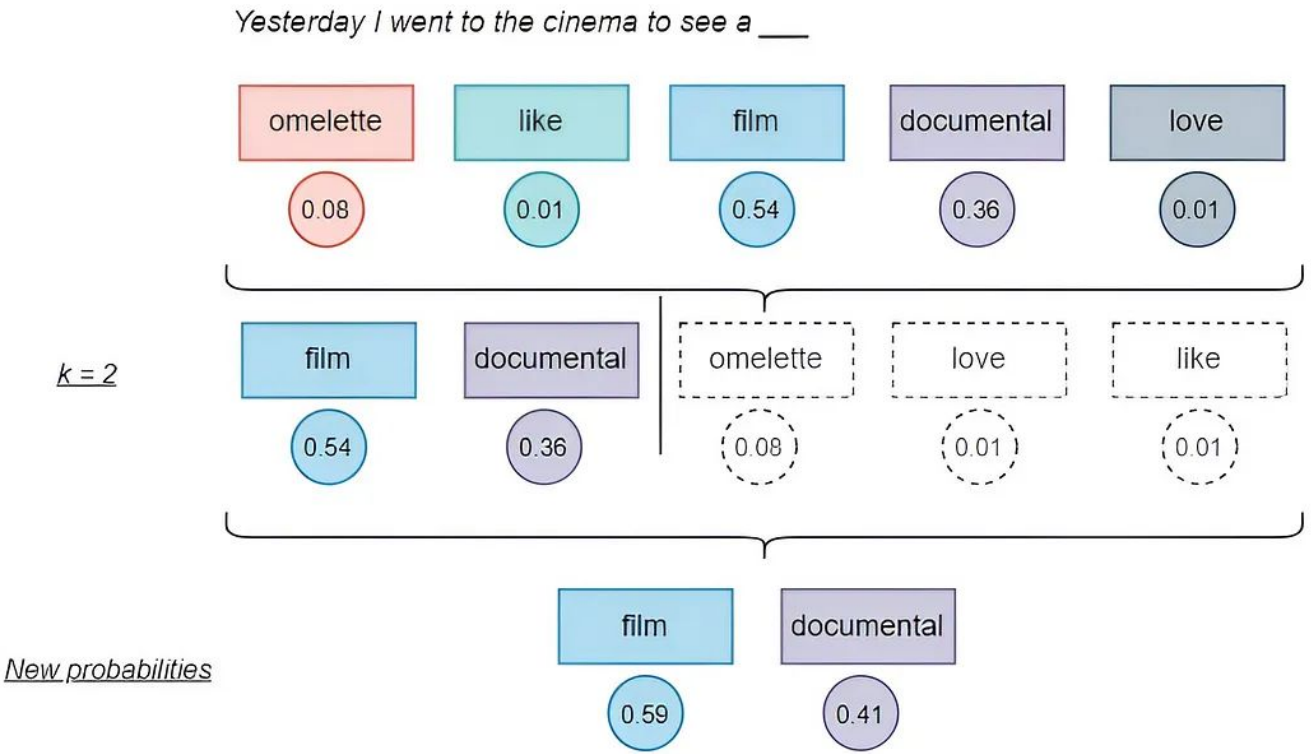
Source: https://www.baeldung.com/cs/softmax-temperature

top_p



Read more: https://medium.com/@daniel.puenteviejo/the-science-of-control-how-temperature-top-p-and-top-k-shape-large-language-models-853cb0480dae

top_k


Yesterday I went to the cinema to see a ___
omelette
like
film
documental
love
0.08
0.01
0.54
0.36
0.01
k = 2
film
documental
omelette
love
like
0.54
0.36
0.08
0.01
0.01
New probabilities
film
documental
0.59
0.41

Then you combine temperature, top_k and top_p

- temperature would influence the distribution
- top_p would select the top most probable tokens (for example, with cumulative 95% probability)
- top_k would select the top K tokens from that

top_p and top_k can be used separately as well

Limitations

- Tokenization
- Alignment
- Adversarial Attacks
- Math
- Training data biases
- Hallucinations
- Computation Limits
- License

Tokenization

- depends on source dataset
- hard to extend to other languages
- commonly heavily-optimized for English language
- hard to change later

Alignment

- “As an AI language model, I”
- Hard to overcome from hosted provider, gets fixed quickly
- There are local models with overcomed alignment, like “Dolphin” and “Hermes” type of models

An overview of threats to

An attacker attempts to *indirectly* prompt LLMs integrated in applications

Injection Method

- **Passive methods** (by retrieval)
- **Active methods** (e.g., emails)
- **User-driven injections**
- **Hidden injections**

Affected parties

- **End-users**
- **Developers**
- **Automated systems**
- **The LLM itself** (availability)

Threats

Information Gathering	Fraud	Intrusion	Malware	Manipulated Content	Availability
• Personal data • Credentials • Chat leakage	• Phishing • Scams • Masquerading	• Persistence • Remote control • API calls	• Spreading injections *(Prompts as worms)* • Spreading malware	• Wrong summary • Disinformation • Propaganda/bias • Data hiding • Ads/promotion	• DoS • Increased computation

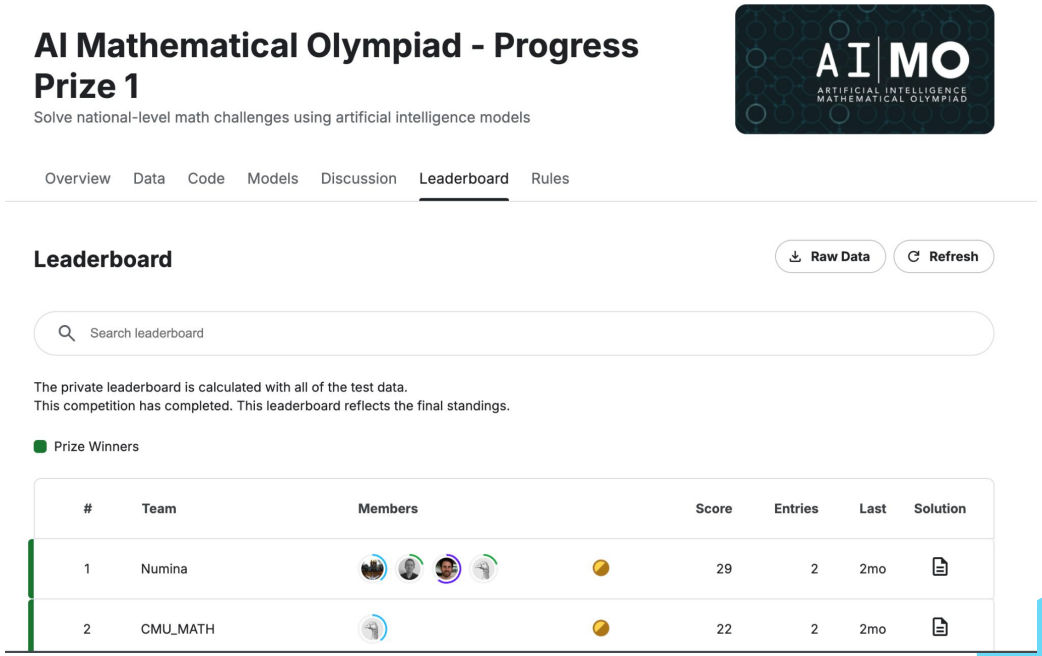
Types of Adversarial Attacks

Attack	Type	Description
Token manipulation	Black-box	Alter a small fraction of tokens in the text input such that it triggers model failure but still remain its original semantic meanings.
Gradient based attack	White-box	Rely on gradient signals to learn an effective attack.
Jailbreak prompting	Black-box	Often heuristic based prompting to "jailbreak" built-in model safety.
Human red-teaming	Black-box	Human attacks the model, with or without assist from other models.
Model red-teaming	Black-box	Model attacks the model, where the attacker model can be fine-tuned.

https://lilianweng.github.io/posts/2023-10-25-adv-attack-llm/

Math

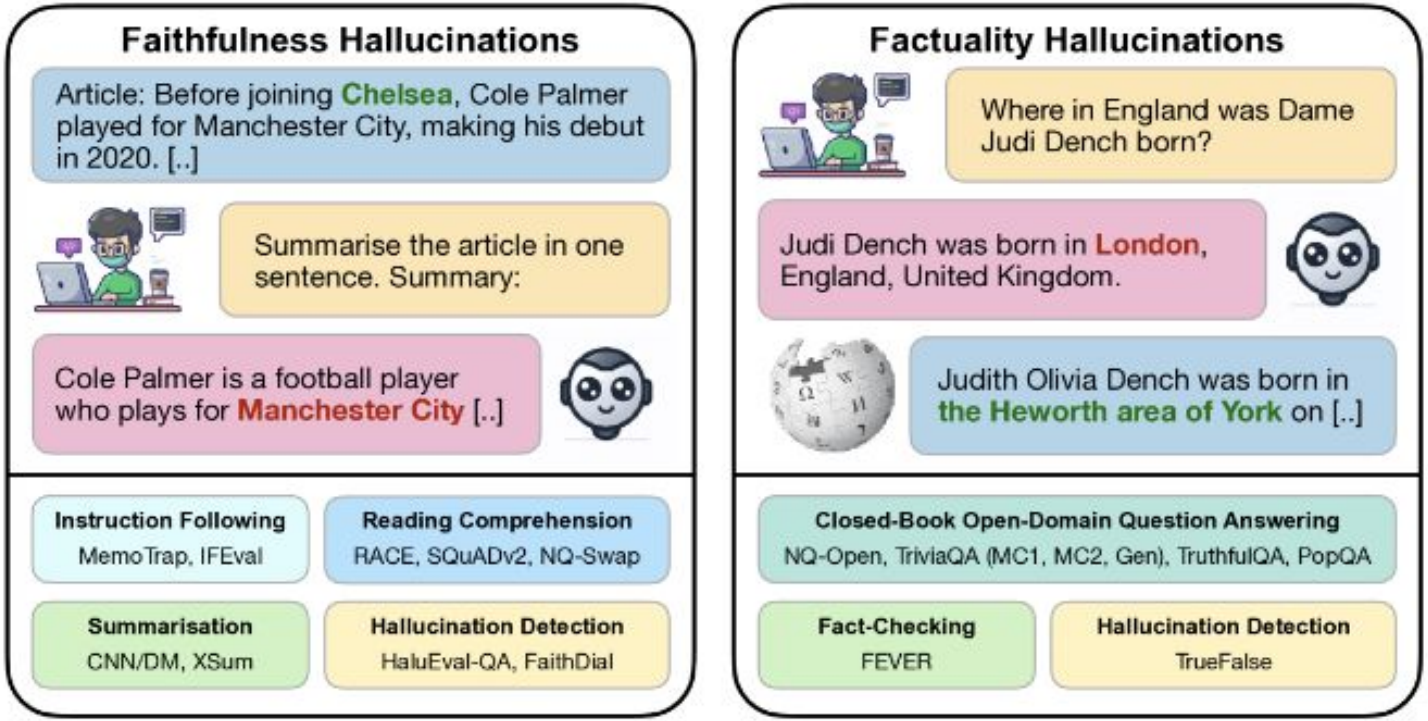
- models are bad at math
- they have no notion of computer calculation
- tokenizer harms math performance



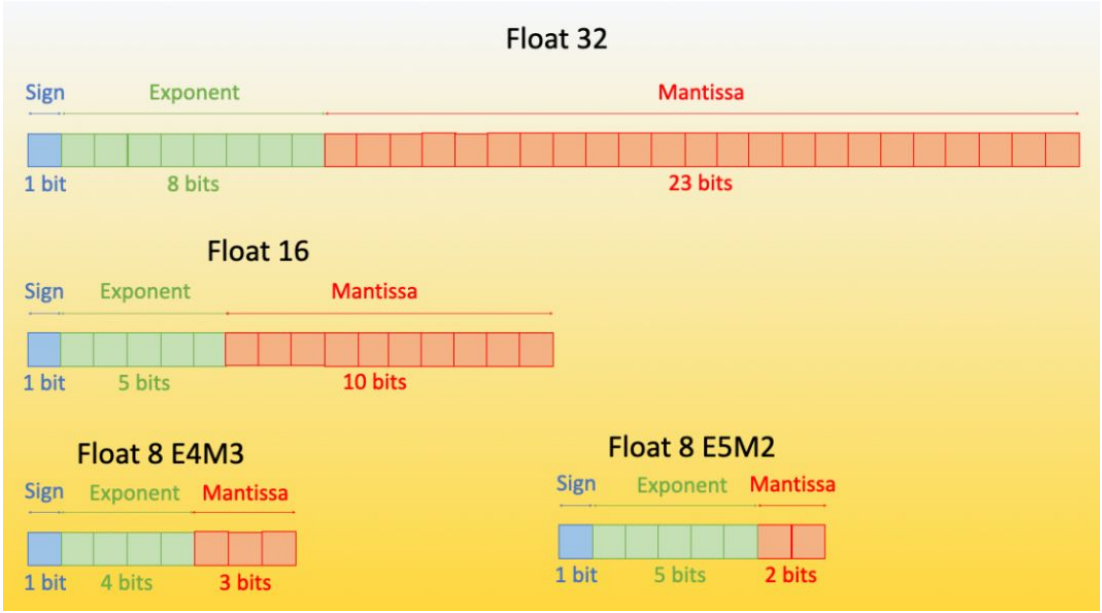
Training data biases and more

no possible way to check original dataset for misinformation

Hallucinations



Available precisions



+ bfloat16

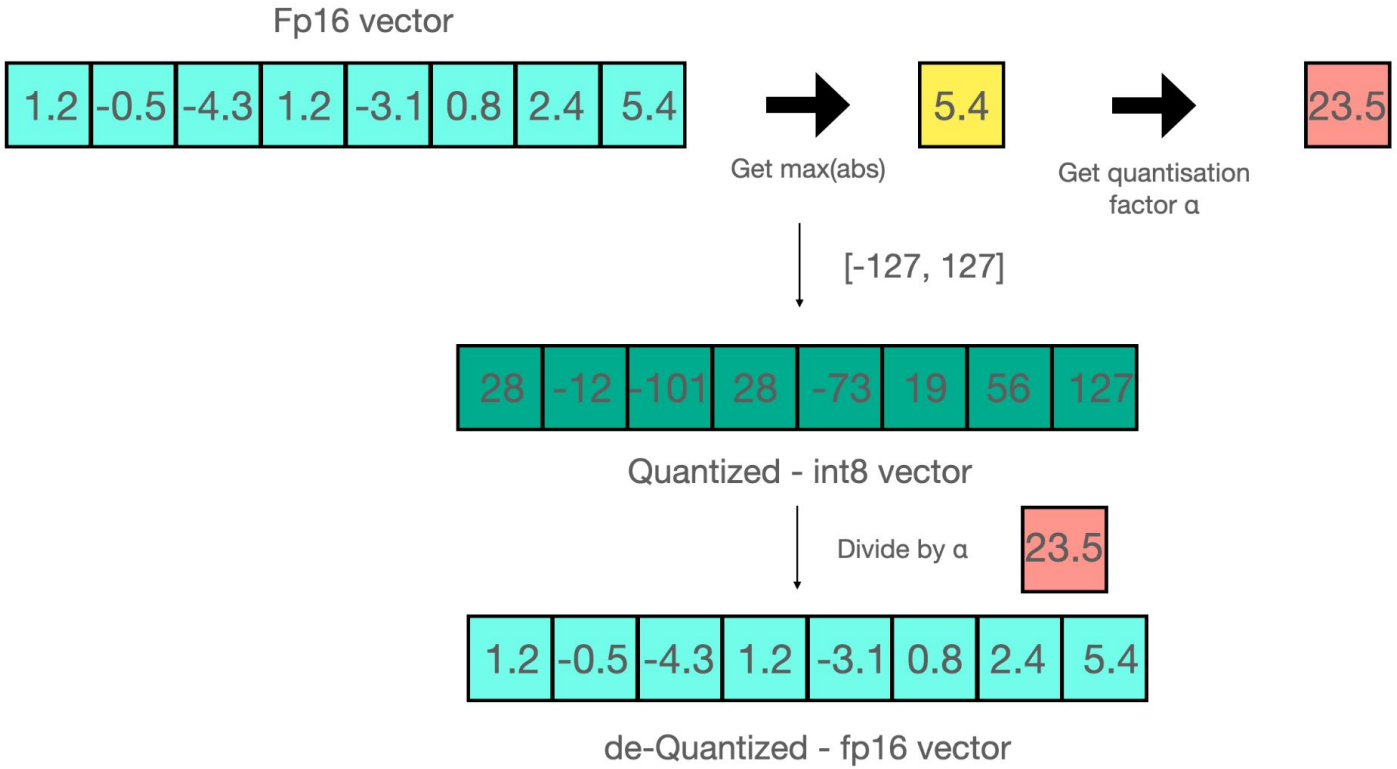
1 sign, 8 exponent, 7 mantissa

It's a "Brain" 🧠 float (Google Brain)

https://huggingface.co/blog/hf-bitsandbytes-integration

https://newsletter.theaiedge.io/p/float32-vs-float16-vs-bfloat16

How quantization works



How does bitsandbytes work

1. From the input hidden states, extract the outliers (i.e. values that are larger than a certain threshold) by column.
2. Perform the matrix multiplication of the outliers in FP16 and the non-outliers in int8.
3. Dequantize the non-outlier results and add both outlier and non-outlier results together to receive the full result in FP16.

Quantization

Use lower precision:

- float16
- bfloat16
- float8 (NVIDIA-only)
- int8
- int4

Anything below bfloat16 leads to performance degradation!

Quantization

Tradeoffs

1. Precision vs. Performance
   Nearly the same result, 15-30% slower in 4bit.
   On-the-fly is slower for fp8 as well, but it would be faster if you pre-quantize:
   https://docs.vllm.ai/en/latest/quantization/fp8.html#offline-quantization-with-static-activation-scaling-factors
2. Inference Speed vs. Hardware Cost
   3090 - 24 GB - 500$ used
   4090 - 24 GB - 2300$ new
   A6000 - 48 GB - 7000$ new

Local Inference

- Transformers
- VLLM
- text-generation-inference
- llama.cpp
- llamafile

Local Inference: transformers

- supports loading hundreds of models from the Huggingface hub
- supports hundreds of tasks
- can be inefficient due to flexibility

https://github.com/huggingface/transformers

Local Inference: VLLM

- State-of-the-art serving throughput
- Efficient management of attention key and value memory with PagedAttention
- Continuous batching of incoming requests
- Fast model execution with CUDA/HIP graph
- Quantizations: GPTQ, AWQ, INT4, INT8, and FP8.
- Optimized CUDA kernels, including integration with FlashAttention and FlashInfer.
- Speculative decoding
- Chunked prefill

https://github.com/vllm-project/vllm

Local Inference: VLLM

- Both online and offline inference
- Various optimized CUDA kernels for different use cases

https://github.com/vllm-project/vllm

Local Inference: vLLM

```python
from vllm import LLM, SamplingParams

prompts = [
    "Hello, my name is",
    "The president of the United States is",
    "The capital of France is",
    "The future of AI is",
]
sampling_params = SamplingParams(temperature=0.8, top_p=0.95)

llm = LLM(model="facebook/opt-125m", quantization="fp8")

outputs = llm.generate(prompts, sampling_params)

for output in outputs:
    prompt = output.prompt
    generated_text = output.outputs[0].text
    print(f"Prompt: {prompt!r}, Generated text: {generated_text!r}")
```

Local Inference: text-generation-inference

- Online inference-focused framework
- Supports out of the box most of the models on HuggingFace with different options
- OpenAI-compatible server

https://github.com/huggingface/text-generation-inference

Local Inference: llama-cpp-python

- wrapper around llama.cpp
- heavily-optimized library for inference
- GGUF format supports quantizations starting from 2-bit
- supports CUDA, OpenCL, Metal, Vulkan and more accelerators
- contains code for optimized CPU inference
- can be hosted as OpenAI-compatible web server, so other AI libraries are compatible
- can make decoder model an embedding model

https://github.com/abetlen/llama-cpp-python

Local Inference: llamafile

- CPU-focused implementation with single executable file
- LLaMa 70B 50 tok/s with AMD 5600X + 128 GB RAM

https://github.com/Mozilla-Ocho/llamafile

FlashAttention

- attention has quadratic cost, you have to attend all tokens
- flash attention use custom CUDA kernels to make it log(N)
- Ampere, Ada, or Hopper GPUs (e.g., A100, RTX 3090, RTX 4090, H100).
- Datatype fp16 and bf16 (bf16 requires Ampere, Ada, or Hopper GPUs).
- All head dimensions up to 256. Head dim > 192 backward requires A100/A800 or H100/H800. Head dim 256 backward now works on consumer GPUs (if there's no dropout) as of flash-attn 2.5.5.
- doesn’t run with sliding attention (Mistral family)

Can I use CPU?

- Yes, but it’s optimized for sequential operations, not for parallel.
- Different memory allocation paradigm (writes can be fast at arbitrary cell, GPUs write to memory in blocks)
- Limited amount of parallel cores and lack of instructions for parallelism

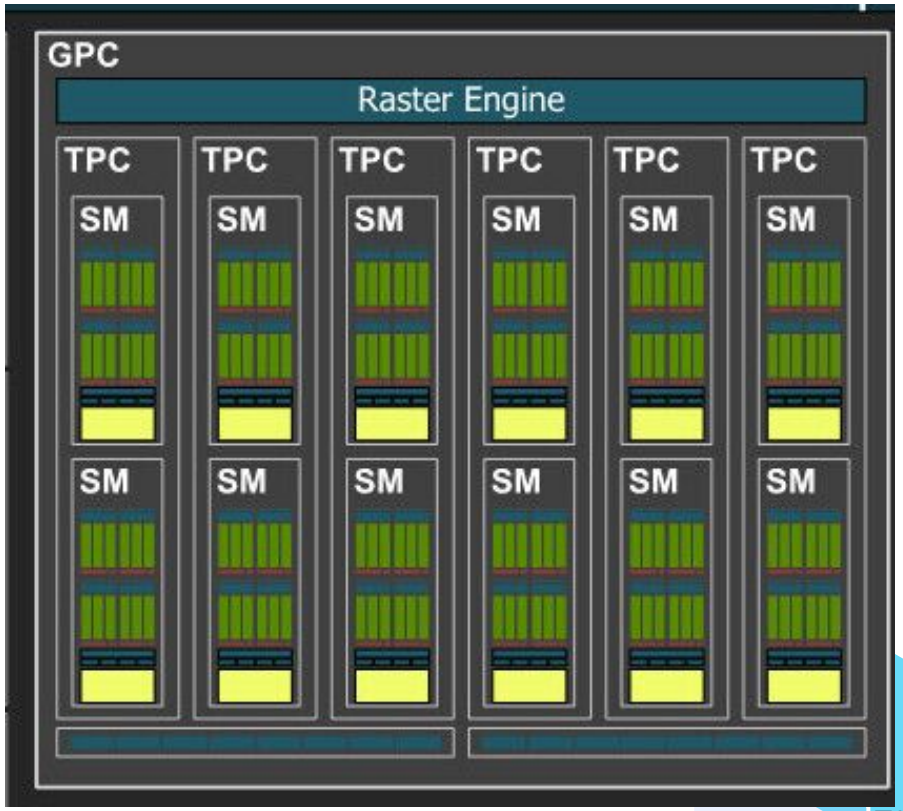
Patrick as a CPU

4090/RTX A6000 Ada block

https://www.techpowerup.com/299130/nvidia-ada-ad102-block-diagram-and-new-architectural-features-detailed

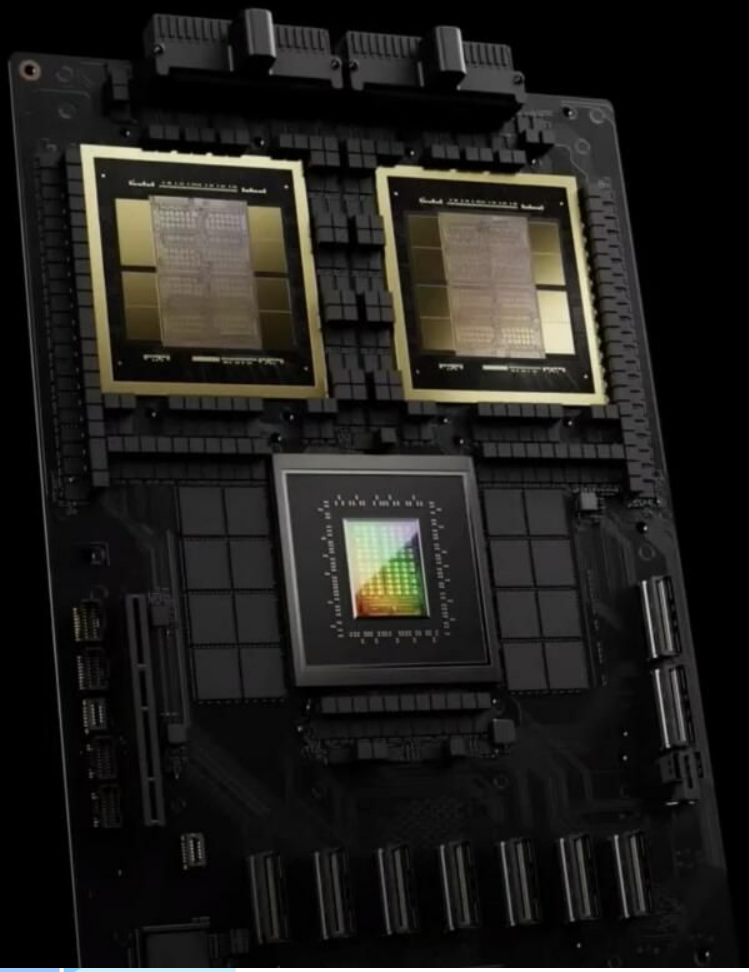
Graphics processing clusters (rendering/calculation)

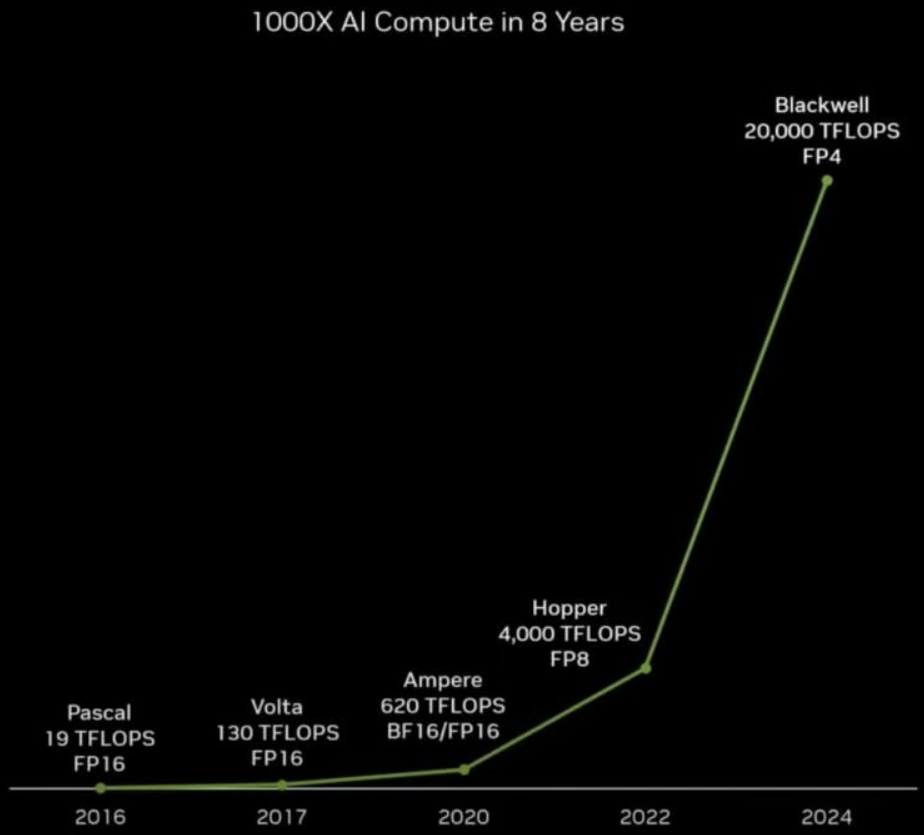
Texture processing clusters - a group of streaming processors.
Raster engine for 2D processing (not relevant for us)



Streaming multiprocessors

Main number crunching component.
Different precisions have different throughput (lower is faster).
Tensor cores are in blocks of 8, so please pad your batches to nearest 8. (pad_to_multiple_of in .generate)




1000X AI Compute in 8 Years
Pascal
19 TFLOPS
FP16
Volta
130 TFLOPS
FP16
Ampere
620 TFLOPS
BF16/FP16
Hopper
4,000 TFLOPS
FP8
Blackwell
20,000 TFLOPS
FP4
2016
2017
2020
2022
2024

Remember this slide?

Different precisions have different throughput (lower is faster).

More correct term is “could more efficiently saturate memory bandwidth” (roughly 2x each level)

LIMITER ANALYSIS

Lesson 1: Understand your performance limiters

Math limited if: $\frac{FLOPS}{bytes} > \frac{math\ throughput}{memory\ bandwidth}$

Left metric is algorithmic mix of math and memory ops called arithmetic intensity

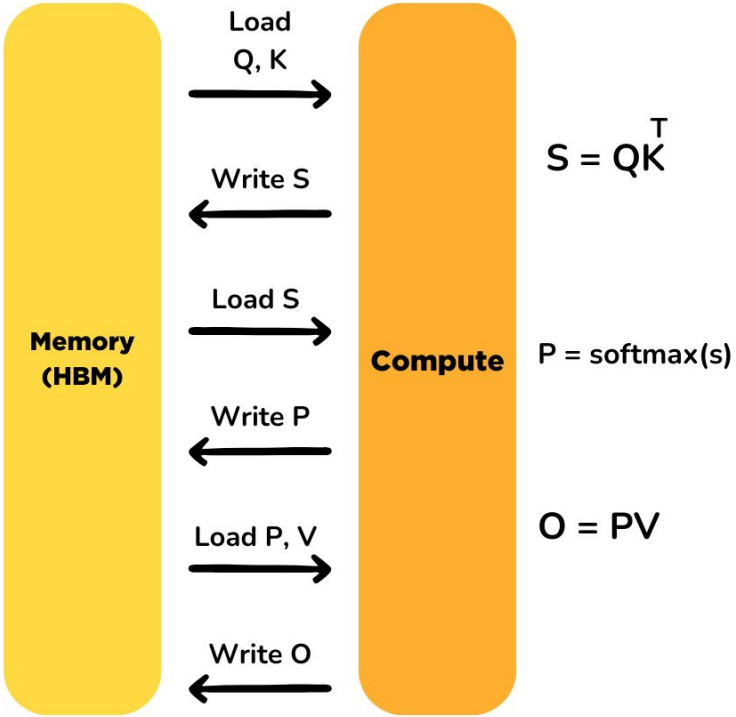
Right metric is the processor’s ops/byte ratio - e.g. V100 can execute 125/0.9=139 FLOPS/B

Comparing arithmetic intensity to ops/byte ratio indicates what algorithm is limited by!

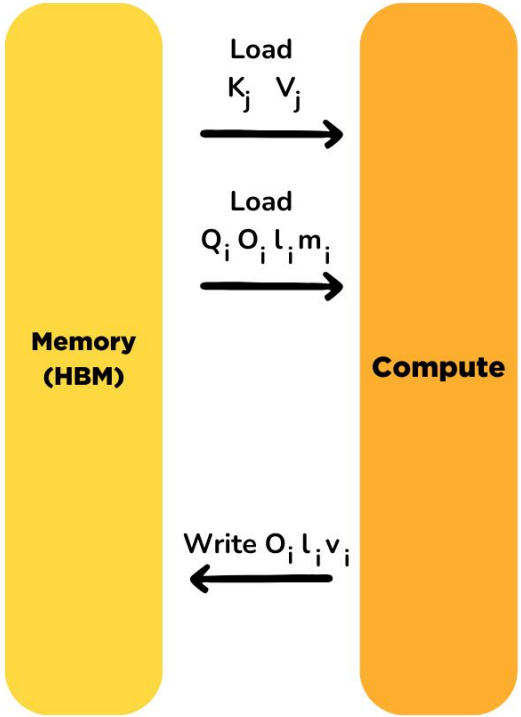
Operation	Arithmetic Intensity	Limiter
Residual addition	0.166	Memory
ReLU activation	0.25	Memory
Batch normalization	O(10)	Memory
Convolution	1-10000+	Memory/Math

(assumes FP16 data)

Standard Attention Implementation



Flash Attention



Kernel operations fused together, reducing reads & writes

$S_{ij} = Q_i K_j^T$
$\tilde{m}$ = rowmax of S
P = exp(s - m)
l = rowsum of P
$m = \max(\tilde{m}_{ij}, m)$
calculate O from l & m

Initialize O, l and m matrices with zeroes. m and l are used to calculate cumulative softmax. Divide Q, K, V into blocks (due to SRAM's memory limits) and iterate over them, for i is row & j is column.

Deployment use cases

- Offline Inference
- Online Inference

Offline Inference

- Batch processing of large datasets
- No real-time constraints
- Optimized for throughput

Examples: Data analysis, periodic tasks

VLLM, transformers.

Online Inference

- Real-time processing of individual requests
- Low-latency requirements
- Optimized for quick response times
- Examples: User-facing applications, real-time decision making

VLLM, TGI, transformers

Scaling inference

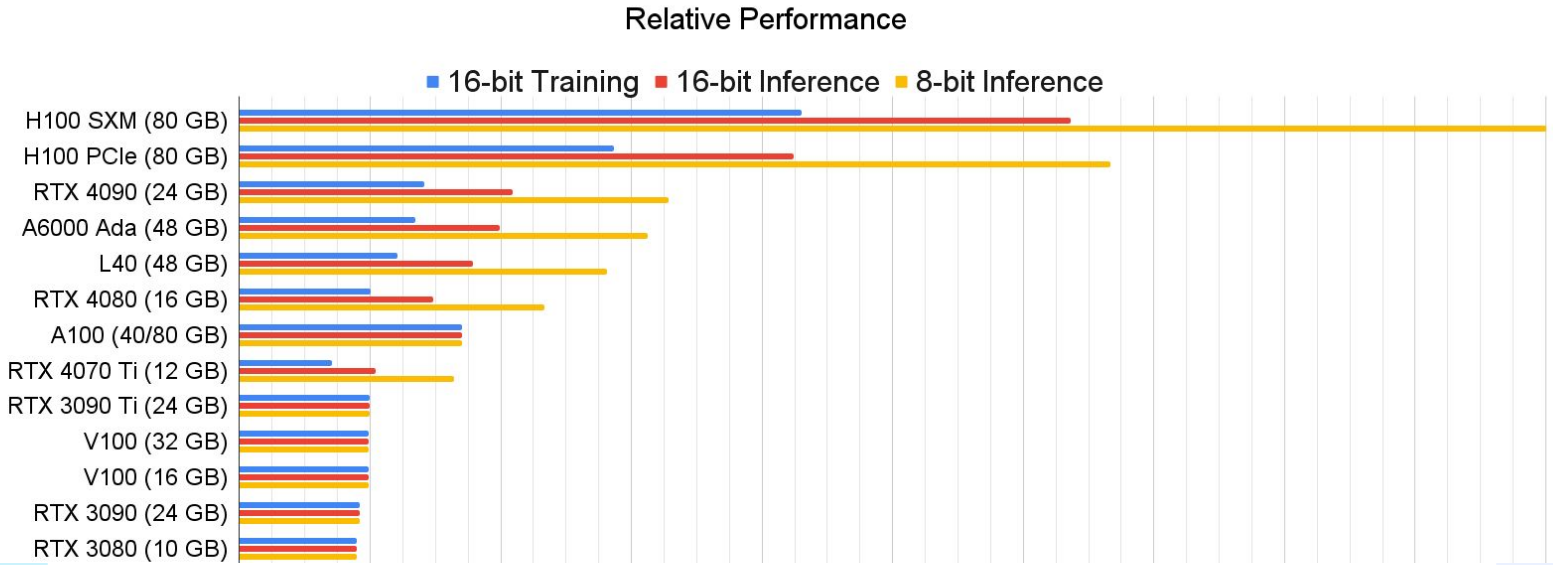
Good interconnect!

Your links:

1. PCIe Lane. (does your CPU have enough PCIe lanes?)
2. Interconnect between PCIe (first question as well)
3. Interconnect between servers (is it enough for network traffic and inter-GPU traffic?)

what hardware?

Read more: https://timdettmers.com/2023/01/30/which-gpu-for-deep-learning/



what hardware?

Read more: https://timdettmers.com/2023/01/30/which-gpu-for-deep-learning/

- NVIDIA only
- Ampere+ generation
- fp8/int8
- memory bandwidth
- 24 GB+ (please use professional)

Same generation, 4090 (24 GB) vs A6000 Ada (48 GB)
VLLM fp8 Mistral-7B-0.1 400 tok/s vs 2000 tok/s prefill

Thanks!

Do you have any questions?